

Fundamentals of Deep Learning and Programming

Lecture 7: Introduction to Natural Language Processing (NLP)

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Python List

```
text = "I LIKE CATS"
print(text.lower())
print(text.lower().split())
```

```
i like cats
['i', 'like', 'cats']
```

```
toks = ["How", "are", "you?"]
text = ':'.join(toks)
print(text)
```

```
text = ' '.join(toks)
print(text)
```

```
How:are:you?
How are you?
```

Python Dictionary

```
person = {"name": "Minsoo", "age": 25}
```

```
print(person.get("name")) # Minsoo
print(person.get("job")) # None (because "job" doesn't exist)
print(person.get("job", "Not Allowed")) # default value is returned
```

```
Minsoo
None
Not Allowed
```

```
person = {"name": "Minsoo", "age": 25}
```

```
print(person["name"])
print(person["job"])
```

```
Traceback (most recent call last):
  File "D:\dev\python\myTorch\pytorch\lec\c07\python test.py", line 8, in <module>
    print(person["job"])
KeyError: 'job'
```

Regular Expression

Regular Expression (1)

```
import re

text = "I like cats and dogs."

# find all words
# \w = letters, numbers, underscore
words = re.findall(r"\w+", text)
print(words)
```

```
['I', 'like', 'cats', 'and', 'dogs']
```

regular_ex1.py

re.findall(...)

- **re** is Python's regular expression module.
- **re.findall(pattern, text)** searches through a string in text and returns a list of all non-overlapping matches of the pattern.

r"\w+"

- r"..." means a raw string literal, so backslashes are treated literally.
- **\w** is a regex token meaning “**word character**”.
 - It matches letters (A–Z, a–z), digits (0–9), and the underscore _.
- **+** means “one or more” consecutive occurrences.

Word character in Regular Expression

In regular expressions, a “word character” is any character that belongs to this set:

- **Uppercase letters:** A–Z
- **Lowercase letters:** a–z
- **Digits:** 0–9
- **Underscore:** _

In regex, it is represented by: `\w`

Examples of **word characters**

- A, b, Z
- 7, 0, 4
- _ (underscore)

Examples of characters that are **not** word characters

- Spaces: " "
- Punctuation: , . ! ? : ;
- Symbols: @ # \$ % ^ & *

Regular Expression (2)

```
import re

text = "Hello, world! How are you?"

# find all punctuation
punct = re.findall(r"[.,!?]", text)
print(punct)
```

```
['.', '!', '?']
```

"[.,!?]" — character class

- Square brackets [...] define a **character class**:
- Match one character from the list inside the brackets.

Regular Expression (3)

```
import re

text = "Hello, world! How are you?"

# find all punctuation
punct = re.findall(r"[^\w\s]", text)
print(punct)
```

```
['!', ',', '?']
```

Square brackets [...] define a **character class**:

Match one single character from the set inside the brackets.

Example:

[abc] matches a, b, or c.

When ^ appears as the first character inside [...], it means:

“Match any character NOT in this set.”

r"[^\w\s]": Match any character that is NOT \w and NOT \s.

\w — word character

- letters A–Z, a–z
- digits 0–9
- underscore _
- AND all Unicode letters (ex: Korean, Greek, etc.).

\s — whitespace

- space " "
- tab \t
- newline \n
- other whitespace characters

Regular Expression (4)

```
import re

text = "Hello, world! How are you?"

tokens = re.findall(r"[A-Za-z0-9]+|[\sA-Za-z0-9]", text)
print(tokens)
```

```
['Hello', ',', 'world', '!', 'How', 'are', 'you', '?']
```

`[A-Za-z0-9]+`

→ one or more letters or digits

→ matches: Hello, world, How, are, you

`[\sA-Za-z0-9]`

→ any ONE character that is not:

- whitespace (`\s`)
- letter
- digit

→ matches **punctuation**: , ! ?

Regular Expression (5)

```
import re

text = "I like cats !"

#splits words and punctuation
tokens = re.findall(r"[A-Za-z0-9]+|[\sA-Za-z0-9]", text)

print(tokens)
```

```
['I', 'like', 'cats', '!']
```

`[A-Za-z0-9]+`

one or more letters/digits → word tokens

`[\sA-Za-z0-9]`

one character that is **not** whitespace or alphanumeric → punctuation/symbol

|
OR

```
import re

text = "I have a cat and a dog."

tokens = re.findall(r"cat|dog", text)

print(tokens)
```

```
['cat', 'dog']
```

Regular Expression (6)

```
import re

# We want to turn this into "cats!"
# (remove the space before punctuation).
text = "I like cats !"

clean = re.sub(r"\s+([!?.,])", r"\1", text)
print(clean)
```

I like cats!

`re.sub(pattern, repl, string)`

- pattern: what we're looking for (a regex)
- repl: what we substitute it with
- string: the original text

`(...)` parentheses create a capturing group

`[!?.,]` is a **character class**:

It matches exactly one of these characters: !, ?, ., ,

`r"\1"`

- `\1` is a **backreference** to capturing **group 1**.
- Group 1 is whatever was matched inside the first (...).

Counter class inside collection module

```
from collections import Counter
```

- Counter class is like a dictionary that counts how many times each item appears.

Counting words in a list

```
from collections import Counter

counter = Counter()          # create an empty counter
counter.update(["cat", "dog", "cat"])

print(counter)
```

```
Counter({'cat': 2, 'dog': 1})
```

counter.update() adds counts to the counter.

Updating multiple times

```
from collections import Counter

counter = Counter()          # create an empty counter
counter.update(["cat", "dog", "cat"])
counter.update(["cat", "cow"])

print(counter)
```

```
Counter({'cat': 3, 'dog': 1, 'cow': 1})
```

Updating with characters of a string

```
from collections import Counter

counter = Counter()          # create an empty counter
counter.update("hello")

print(counter)
```

```
Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})
```

```
from collections import Counter

counter = Counter()          # create an empty counter
texts = ["I like cats.", "I like dogs"]

for text in texts:
    tokens = text.lower().split()
    counter.update(tokens)

print(counter)
```

```
Counter({'i': 2, 'like': 2, 'cats.': 1, 'dogs': 1})
```

Counting words

```
from collections import Counter

tokens = ["cat", "dog", "cat", "bird", "cat", "dog"]
counter = Counter(tokens)

print(counter.most_common()) # sorts them by frequency.
[('cat', 3), ('dog', 2), ('bird', 1)]
```

`counter.most_common()`
Return all items sorted by frequency (high → low)

`counter.most_common(n)` Return only the top `n` items

```
from collections import Counter

text = "I like dogs and I like cats and I like animals"
words = text.lower().split() # very simple tokenization

counter = Counter(words)

for word, freq in counter.most_common():
    print(word, "→", freq)
```

```
i → 3
like → 3
and → 2
dogs → 1
cats → 1
animals → 1
```

```
from collections import Counter

animals = "catdogcatcat"
counter = Counter(animals)

print(counter.most_common(1))
```

```
[('c', 3)]
```

Full NLP pipeline usually looks like this

1. Tokenize
2. Build vocabulary
3. Encode text into token IDs
4. → Word embeddings (required for ML models)
5. → Feed embeddings into a model (Transformer, LSTM, etc.)
6. Train
7. Decode prediction back to text

1 Pre-processing (Breaking down)

Tokenization

: breaking sentences into smaller, meaningful units called tokens.

These tokens can be words, subwords, or even characters, depending on how the model is designed.

Text : “I like cats”

Tokens : [“I”, “like”, “cats”]

Why tokenization is important

Computers don't understand raw text — they need a structured way to process it.

Tokenization helps the computer

- Understand **where one word ends and another begins**
- Handle **special cases** like punctuation or compound words
- Prepare text for embedding (= turning tokens into numbers)

How can a computer understand text like 'I like cats'?

- Tokenization is the first step: **breaking text into pieces (tokens)** that can be mapped to numbers.
- The **tokenizer** defines what the model "sees" as a **single meaningful unit**.

Text: "I like cats."

Tokens: ["I", "like", "cats", "."]

IDs: [12, 45, 91, 7]

Our simple code for tokenization

- Splits text into **tokens** (words, punctuation)
- Builds a **vocabulary** (tok2id, id2tok)
- Converts tokens into **integer IDs** (encode)
- Turns IDs back to text (decode)

Simple Tokenizer

```
# (building vocabulary → encode → decode)
```

```
import re
from collections import Counter
```

```
PAD, UNK, BOS, EOS = "<pad>", "<unk>", "<bos>", "<eos>"
```

```
# Splits text into tokens (words, numbers, punctuation)
```

```
def tokenize(text, lowercase=True):
```

```
    if lowercase:
        text = text.lower()
```

```
    # Split on whitespace, keep punctuation as separate tokens
```

```
    return re.findall(r"[A-Za-z0-9]+|[\^\sA-Za-z0-9]", text)
```

```
# Scans a corpus, counts tokens, and creates
```

```
# tok2id/id2tok with special tokens
```

```
def build_vocab(texts, min_freq=1, lowercase=True):
```

```
    counter = Counter()
```

```
    for t in texts:
```

```
        counter.update(tokenize(t, lowercase))
```

```
    # places the special tokens at the beginning of the vocabulary
```

```
    id2tok = [PAD, UNK, BOS, EOS]
```

```
    for tok, freq in counter.most_common():
```

```
        if freq >= min_freq and tok not in id2tok:
```

```
            id2tok.append(tok)
```

```
    tok2id = {t: i for i, t in enumerate(id2tok)}
```

```
    return tok2id, id2tok
```

These are special tokens commonly used in NLP:

- <pad> : padding token
- <unk> : unknown token (words not in vocabulary)
- <bos> : beginning of sentence
- <eos> : end of sentence

`build_vocab()` reads all texts in corpus, tokenizes them, counts token frequencies, and creates:

Build lookup dictionary

tok2id: token → ID (dictionary)

id2tok: ID → token (list)

Explanation of code

```
# ---- simple demo ----
if __name__ == "__main__":
    corpus = [
        "I like cats.",
        "I like dogs a lot!",
        "Cats are cute; dogs are loyal."
    ]
    tok2id, id2tok = build_vocab(corpus, min_freq=1, lowercase=True)
```

```
def build_vocab(texts, min_freq=1, lowercase=True):
    counter = Counter()
    for t in texts:
        counter.update(tokenize(t, lowercase))

    print(counter)
```

```
Counter({'i': 2, 'like': 2, 'cats': 2, '.': 2, 'dogs': 2, 'are': 2, 'a': 1, 'lot': 1, '!': 1, 'cute': 1, ';': 1, 'loyal': 1})
```

```
# places the special tokens at the beginning of the vocabulary
id2tok = [PAD, UNK, BOS, EOS]
for tok, freq in counter.most_common():
    if freq >= min_freq and tok not in id2tok:
        id2tok.append(tok)
print(id2tok)
```

```
['<pad>', '<unk>', '<bos>', '<eos>', 'i', 'like', 'cats', '.', 'dogs', 'are', 'a', 'lot', '!', 'cute', ';', 'loyal']
```

```
tok2id = {t: i for i, t in enumerate(id2tok)}
print(tok2id)
```

```
{'<pad>': 0, '<unk>': 1, '<bos>': 2, '<eos>': 3, 'i': 4, 'like': 5, 'cats': 6, '.': 7, 'dogs': 8, 'are': 9, 'a': 10, 'lot': 11, '!': 12, 'cute': 13, ';': 14, 'loyal': 15}
return tok2id, id2tok
```

build_vocab() reads all texts in corpus, tokenizes them, counts token frequencies, and creates:

Build lookup dictionary

tok2id: token → ID (dictionary)

id2tok: ID → token (list)

Simple Tokenizer

```
def encode(text, tok2id, add_specials=True, lowercase=True):  
    toks = tokenize(text, lowercase)  
    ids = [tok2id.get(t, tok2id[UNK]) for t in toks]  
  
    if add_specials:  
        ids = [tok2id[BOS]] + ids + [tok2id[EOS]]  
    return ids # wrap with torch.tensor(ids) if you want tensors
```

toks

['i', 'really', 'like', 'cats', '!']

ids

[4, 1, 5, 6, 12]

ids

[2, 4, 1, 5, 6, 12, 3]

tok2id

{'<pad>': 0, '<unk>': 1, '<bos>': 2, '<eos>': 3, 'i': 4, 'like': 5, 'cats': 6, '.': 7, 'dogs': 8, 'are': 9, 'a': 10, 'lot': 11, '!': 12, 'cute': 13, ';': 14, 'loyal': 15}

---- simple demo ----

if __name__ == "__main__":

corpus = [

"I like cats.",

"I like dogs a lot!",

"Cats are cute; dogs are loyal."

]

tok2id, id2tok = build_vocab(corpus, min_freq=1,
 lowercase=True)

s = "I really like cats!"

ids = encode(s, tok2id, add_specials=True,
 lowercase=True)

print("Encoded:", ids)

Encoded: [2, 4, 1, 5, 6, 12, 3]

Simple Tokenizer

id2tok

['<pad>', '<unk>', '<bos>', '<eos>', 'i', 'like', 'cats', '.', 'dogs', 'are', 'a', 'lot', '!', 'cute', ';', 'loyal']

```
def decode(ids, id2tok, strip_specials=True):
    toks = [id2tok[i] for i in ids if 0 <= i < len(id2tok)]

    if strip_specials:
        toks = [t for t in toks if t not in {PAD, BOS, EOS}]

    # Simple detokenization:
    # join then fix space before punctuation
    text = " ".join(toks)
    text = re.sub(r"\s+([.!,?;:])", r"\1", text)
    return text
```

ids [2, 4, 1, 5, 6, 12, 3]

toks ['<bos>', 'i', '<unk>', 'like', 'cats', '!', '<eos>']

toks ['i', '<unk>', 'like', 'cats', '!']

Optionally remove special tokens (except <unk>).

Remove the space before punctuation.

Example: "cats !" → "cats!"

tok2id

{'<pad>': 0, '<unk>': 1, '<bos>': 2, '<eos>': 3, 'i': 4, 'like': 5, 'cats': 6, '.': 7, 'dogs': 8, 'are': 9, 'a': 10, 'lot': 11, '!': 12, 'cute': 13, ';': 14, 'loyal': 15}

```
# ---- simple demo ----
if __name__ == "__main__":
    corpus = [
        "I like cats.",
        "I like dogs a lot!",
        "Cats are cute; dogs are loyal."
    ]
    tok2id, id2tok = build_vocab(corpus, min_freq=1,
                                lowercase=True)
```

s = "I really like cats!"

ids = encode(s, tok2id, add_specials=True, lowercase=True)

print("Encoded:", ids)

print("Decoded:", decode(ids, id2tok))

Encoded: [2, 4, 1, 5, 6, 12, 3]

Decoded: i <unk> like cats!

Full NLP pipeline usually looks like this

1. Tokenize
2. Build vocabulary
3. Encode text into token IDs
4. → Word embeddings (required for ML models)
5. → Feed embeddings into a model (Transformer, LSTM, etc.)
6. Train
7. Decode prediction back to text

1 Pre-processing (Breaking down)

Embedding : turning words into numbers:

- Because models cannot use IDs directly.
- IDs like 5, 10, 23 have no meaning.
- **Embeddings** convert IDs into **dense vectors** that **contain semantic meaning**.

`nn.Embedding` (inside PyTorch model):

- corpus → tokenize → embeddings (random) → train model → embeddings improve

Adding Embeddings

```
import torch
import torch.nn as nn

id2tok = ['<pad>', '<unk>', '<bos>', '<eos>', 'i', 'like', 'cats',
          '.', 'dogs', 'are', 'a', 'lot', '!', 'cute', ';', 'loyal']
ids = [2, 4, 1, 5, 6, 12, 3]

vocab_size = len(id2tok)
embedding_dim = 16

embedding = nn.Embedding(vocab_size, embedding_dim) # randomly generated

ids = torch.tensor([ids]) # batch of size 1
embedded = embedding(ids)

print(embedded.shape) # (1, seq_len, 16) torch.Size([1, 7, 16])
print(embedded)
```

```
tensor([[[ 0.4808, -1.5227,  0.7144, -0.6227, -0.0741, -0.6725, -0.6258,
          0.2058,  0.7669,  1.0282, -0.3162,  0.1586, -0.0247, -1.4010,
          -0.5281, -0.9687],
        [-0.7281,  1.3209,  0.9018, -0.7682,  0.4608,  0.6335,  0.9130,
          -0.3930,  0.5214,  0.3784, -0.9379,  0.7722, -1.5735,  0.7674,
          -1.8908, -1.5676],
        [-0.5432, -1.9011,  0.6429,  1.5343,  0.3335, -1.2450,  0.3606,
          -0.7542,  0.1833,  1.6124, -0.3908,  0.7508,  1.7599, -0.0191,
          -2.4435,  0.3196],
        [ 1.4872,  0.6443,  0.0578,  1.3112,  0.2208,  0.1083,  0.0510,
          0.4915, -0.2723,  0.9921,  1.8455,  1.4673,  0.7730,  0.5831,
          -0.0475,  0.6474],
        [ 0.8211, -0.6509,  1.8794, -0.2697,  1.0924, -1.1893,  0.5874,
          0.2931, -1.5728,  0.5272,  1.6783, -0.2127,  0.1290, -0.7861,
          -1.1771, -0.6515],
        [-0.7393,  0.9116, -0.2045,  1.0687, -0.8588, -0.0896,  0.3246,
          1.0881,  0.5395, -0.2190,  0.5630, -0.8640, -0.5902, -0.4089,
          -1.4449, -0.4200],
        [-0.1058,  0.3918, -0.0344, -0.8764, -1.8784,  0.2609,  0.5413,
          1.4720, -2.2720,  0.1284,  1.5655,  0.6109,  1.6136, -0.3390,
          0.5380,  1.4432]]], grad_fn=<EmbeddingBackward0>)
```


Word2Vec is a method in Natural Language Processing (NLP) for turning words into vectors of numbers. These vectors are called **word embeddings**.

The main idea is simple:

Words that appear in similar contexts should have similar vectors.

For example, after training Word2Vec on a large text corpus, we might get:

“cat” $\rightarrow$ $[0.31, -0.22, 0.87, \dots]$, “dog” $\rightarrow$ $[0.29, -0.20, 0.83, \dots]$.

The vectors for “cat” and “dog” are close to each other in the vector space, because they are semantically similar words.

Why do we need Word Embedding ?

Neural networks and other machine learning models work with numbers, not with text. So we must convert each word into a numerical representation.

Before Word2Vec, common approaches were:

- **One-hot vectors**: very long, mostly zeros, no notion of meaning.
- **Bag-of-words**: counts word occurrences, but ignores word order and deeper semantics.

Word2Vec solves these problems by learning:

- **dense** vectors (low-dimensional, e.g. 100 or 300 dimensions),
- **meaningful** vectors (geometry reflects similarity and relationships).

Core Idea : Context Predicts Meaning

Word2Vec is based on a famous idea in linguistics, often called the *distributional hypothesis*:

“You shall know a word by the company it keeps.”

This means that the **context** around a word gives strong clues about its meaning.
For example:

- “The cat drinks milk.”
- “The dog drinks water.”

Because “cat” and “dog” appear in similar sentences and with similar surrounding words, Word2Vec will learn similar vectors for them.

Two Main Architectures: CBOW and Skip-gram

Word2Vec usually refers to two related neural network models:

1. CBOW (Continuous Bag of Words)

Predict the center word from its context words. For example, from “the ____ drinks milk”, predict “cat”.

2. Skip-gram

Predict the context words from a center word. For example, from “cat”, predict words like “the” and “drinks”.

Both models are small neural networks with one hidden layer. The important part is that during training, the network learns good embeddings for each word.

What Does Word2Vec Learn ?

Mathematically, Word2Vec learns an **embedding matrix**:

$$W \in \mathbb{R}^{V \times D},$$

where:

- V is the vocabulary size (number of distinct words),
- D is the embedding dimension,
- each row W_w is the embedding vector for word w .

This matrix is the main result of training. We do not usually use the whole Word2Vec model later, we just use the learned matrix W .

Example of Semantic Structure

One of the famous properties of Word2Vec embeddings is that they capture intuitive relationships between words. For example:

$$\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"}).$$

This shows that the model has captured some notion of gender and royalty in the geometry of the vectors.

Training Trick: Negative Sampling

Computing the full probability distribution over a large vocabulary can be expensive. Word2Vec uses a technique called **negative sampling** to make training efficient.

Informally, for each correct (center, context) pair, the model:

- rewards the pair (makes their vectors more similar),
- and chooses a few random “negative” words and pushes their vectors away.

This allows Word2Vec to train on very large corpora quickly.

- Negative sampling makes training efficient while still learning meaningful relationships.

How Word2Vec is Used in Practice

Once Word2Vec has been trained on a large text corpus, we can:

- use the word vectors as input features to other models (e.g., classifiers, RNNs, Transformers),
- measure similarity between words using cosine similarity,
- explore relationships and analogies between words.

Word2Vec is a way to convert words into useful numerical vectors that capture meaning.

What is Word2Vec ?

Word2Vec is a method to turn words into vectors of numbers (embeddings). The main idea is:

If two words appear in similar contexts, their vectors should become similar.

For example, after training on many sentences, we might get:

“cat” $\rightarrow$ $[0.3, -0.2, 0.8, \dots]$, “dog” $\rightarrow$ $[0.29, -0.18, 0.75, \dots]$.

We will focus on a very small toy world with just four words:

$\{\text{cat}, \text{dog}, \text{milk}, \text{water}\},$

and use this to illustrate the math and intuition.

We use the **row convention**: each word embedding is stored in a row of a matrix.

- In the example, “cat” becomes associated with “milk”, and “dog” with “water”.

Tiny Vocabulary and One-Hot Representation

Assume our tiny vocabulary is:

$$0 \rightarrow \text{"cat"}, \quad 1 \rightarrow \text{"dog"}, \quad 2 \rightarrow \text{"milk"}, \quad 3 \rightarrow \text{"water"}.$$

So the vocabulary size is $V = 4$.

Each word can be represented as a one-hot **row** vector of length V :

$$\text{"cat"} \Rightarrow x_{\text{cat}} = [1, 0, 0, 0],$$

$$\text{"dog"} \Rightarrow x_{\text{dog}} = [0, 1, 0, 0],$$

$$\text{"milk"} \Rightarrow x_{\text{milk}} = [0, 0, 1, 0],$$

$$\text{"water"} \Rightarrow x_{\text{water}} = [0, 0, 0, 1].$$

In practice, we rarely create these one-hot vectors explicitly. Instead, we use the index (0, 1, 2, or 3) to select the correct row from an embedding matrix.

Embedding Matrices (input and output)

Word2Vec (Skip-gram) uses two embedding matrices:

$$W \in \mathbb{R}^{V \times D}, \quad W' \in \mathbb{R}^{V \times D},$$

where:

- V is the vocabulary size (here $V = 4$),
- D is the embedding dimension,
- each row W_w is the **input embedding** of word w ,
- each row W'_w is the **output embedding** of word w .

For a center word w_c , its input embedding is:

$$v_c = W_{w_c}.$$

For a context word w_o , its output embedding is:

$$v'_o = W'_{w_o}.$$

Small Toy Example with Dimension $D = 2$

To make things concrete, let us choose a very small embedding dimension $D = 2$. Suppose that at some point during training, the input embedding matrix W and output embedding matrix W' look like this:

$$W = \begin{bmatrix} W_{\text{cat}} \\ W_{\text{dog}} \\ W_{\text{milk}} \\ W_{\text{water}} \end{bmatrix} = \begin{bmatrix} 0.20 & 0.10 \\ 0.25 & 0.05 \\ -0.30 & 0.40 \\ -0.28 & 0.35 \end{bmatrix}, \quad W' = \begin{bmatrix} W'_{\text{cat}} \\ W'_{\text{dog}} \\ W'_{\text{milk}} \\ W'_{\text{water}} \end{bmatrix} = \begin{bmatrix} 0.10 & 0.30 \\ 0.20 & 0.20 \\ -0.40 & 0.50 \\ -0.35 & 0.45 \end{bmatrix}.$$

Then the input embeddings are:

$$v_{\text{cat}} = W_{\text{cat}} = [0.20, 0.10], \quad v_{\text{dog}} = W_{\text{dog}} = [0.25, 0.05],$$

$$v_{\text{milk}} = W_{\text{milk}} = [-0.30, 0.40], \quad v_{\text{water}} = W_{\text{water}} = [-0.28, 0.35],$$

and the output embeddings are:

$$v'_{\text{cat}} = W'_{\text{cat}} = [0.10, 0.30], \quad v'_{\text{dog}} = W'_{\text{dog}} = [0.20, 0.20],$$

$$v'_{\text{milk}} = W'_{\text{milk}} = [-0.40, 0.50], \quad v'_{\text{water}} = W'_{\text{water}} = [-0.35, 0.45].$$

Note that W and W' are not derived from each other. They are separate trainable matrices learned during training.

Small Corpus and Training Pairs

Imagine a tiny corpus with sentences such as:

- “the cat drinks milk”
- “the dog drinks water”
- “the cat likes milk”
- “the dog likes water”

If we only pay attention to the four words {cat, dog, milk, water} and use a small context window, we might extract Skip-gram training pairs like:

(center: “cat”, context: “milk”), (center: “dog”, context: “water”),

Thus, the model should learn that “cat” is strongly associated with “milk”, and “dog” is strongly associated with “water”. Depending on the corpus, it may also learn that both “milk” and “water” are *drinks*, and that “cat” and “dog” are both *animals*.

Dot Products for Some Pairs

Consider the pair (center = “cat”, context = “milk”). We have:

$$v_{\text{cat}} = [0.20, 0.10], \quad v'_{\text{milk}} = [-0.40, 0.50].$$

The dot product is:

$$v_{\text{cat}} \cdot (v'_{\text{milk}})^T = 0.20 \cdot (-0.40) + 0.10 \cdot 0.50 = -0.08 + 0.05 = -0.03.$$

Now consider (center = “dog”, context = “water”):

$$v_{\text{dog}} = [0.25, 0.05], \quad v'_{\text{water}} = [-0.35, 0.45],$$

so

$$v_{\text{dog}} \cdot (v'_{\text{water}})^T = 0.25 \cdot (-0.35) + 0.05 \cdot 0.45 = -0.0875 + 0.0225 = -0.065.$$

During training, the model will adjust the rows of W and W' to make the dot products for *true* (center, context) pairs larger, and the dot products for *false* pairs smaller.

For example, if (“cat”, “water”) is rarely a real pair in the data, then the model will push

$$v_{\text{cat}} \cdot (v'_{\text{water}})^T$$

to be relatively small, compared to

$$v_{\text{cat}} \cdot (v'_{\text{milk}})^T.$$

Skip-gram Probability (Conceptual)

In the full softmax version, the probability of a context word w_o given a center word w_c is:

$$p(w_o | w_c) = \frac{\exp(v_c \cdot (v'_o)^T)}{\sum_{w=1}^V \exp(v_c \cdot (v'_w)^T)}.$$

In our small example with $V = 4$ and center word “cat”, the denominator sums over {cat, dog, milk, water}:

$$\sum_{w \in \{\text{cat}, \text{dog}, \text{milk}, \text{water}\}} \exp(v_{\text{cat}} \cdot (v'_w)^T).$$

The training objective encourages $p(\text{“milk”} | \text{“cat”})$ to be high, and similarly $p(\text{“water”} | \text{“dog”})$ to be high.

Negative Sampling (Intuition)

The training objective encourages $p(\text{“milk”} \mid \text{“cat”})$ to be high, and similarly $p(\text{“water”} \mid \text{“dog”})$ to be high.

- one positive context word (e.g., “milk” for “cat”),
- a few negative words sampled from the vocabulary (e.g., “dog”, “water” for “cat”).

The negative sampling loss for one center–context pair (w_c, w_o) is:

$$\mathcal{L} = -\log \sigma(v_c \cdot (v'_o)^T) - \sum_{k=1}^K \log \sigma(-v_c \cdot (v'_{n_k})^T),$$

where $\sigma(x) = \frac{1}{1 + e^{-x}}$ is the sigmoid, and v'_{n_k} are the output embeddings of negative words.

In our toy example, if (“cat”, “milk”) is a positive pair, we might sample “dog” and “water” as negative words. Then training:

- increases $v_{\text{cat}} \cdot (v'_{\text{milk}})^T$,
- decreases $v_{\text{cat}} \cdot (v'_{\text{dog}})^T$ and $v_{\text{cat}} \cdot (v'_{\text{water}})^T$.

What Gets Updated

For each training step involving a center word w_c and context word w_o (with some negative samples), only a few rows are updated:

- W_{w_c} (the center word row in W),
- W'_{w_o} (the context word row in W'),
- $W'_{w_{n_k}}$ for each negative sample.

All other rows remain unchanged in that step. Over many steps, this leads to meaningful embeddings.

Final Word Embeddings

After training, we usually take:

$$v_w = W_w$$

as the final embedding vector for each word w (the row of W).

Sometimes, people also combine W and W' by averaging:

$$v_w = \frac{W_w + W'_w}{2}.$$

In our tiny world, this would give us embeddings for “cat”, “dog”, “milk”, and “water” that reflect their roles in the training sentences: cats with milk, dogs with water, and so on.

Each word is represented by a row in the embedding matrices W and W' .

The Skip-gram model learns these matrices so that center words are good at predicting their context words.

Thank you!

